

14: Concurrency, Async, and Networking

Systems interviews often move from an algorithm to the machinery underneath it: how a lock is implemented, why a queue must be bounded, what an async runtime does while a task waits, or how an HTTP parser can be attacked. The systems modules in `rust-interview-lab` each implement one of these mechanisms, with tests.

This chapter explains the mechanism behind each module and the trade-offs it represents. It starts with a one-page review of fundamentals, then works up from threads and synchronization primitives to async Rust, I/O multiplexing, TCP, HTTP, retries, and consistent hashing. It ends by following one request through all of them.

This chapter covers

- Short definitions of operating system, concurrency, networking, and data structure fundamentals
- Threads, `Send` and `Sync`, and choosing between atomics, locks, and channels
- Mutexes, condition variables, semaphores, a spin lock, and bounded queues
- Atomic memory orderings, a lock-free ring buffer, sharding, and arena allocation
- How futures, wakers, `Pin`, and executors work
- `epoll`, partial writes, TCP connection states, and socket options
- HTTP/1.1 framing and request smuggling, REST semantics, retries, and consistent hashing

The modules this chapter describes:

Module	Concept
problems/threads.rs	Threads, scopes, Send/Sync, atomics, OnceLock
problems/smart_pointers.rs	Box, Rc, Arc, RefCell, Mutex, Weak, Cow
problems/adt_idioms.rs	Newtype, enums with data, typed errors, TryFrom
problems/semaphore.rs	Counting semaphore, permits, RAII release
problems/spin_lock.rs	A lock built from an atomic, memory ordering
problems/bounded_queue.rs	Bounded MPMC queue, backpressure, shutdown
problems/ring_buffer.rs	Lock-free SPSC ring, atomic memory ordering
problems/sharded_cache.rs	Sharding to reduce lock contention
problems/bump_allocator.rs	Alignment, arena allocation
problems/retry.rs	Exponential backoff, jitter, Retry-After
problems/consistent_hash.rs	Virtual-node ring, minimal remapping
problems/async_mini.rs	Future, Waker, Pin, block_on, executor
problems/http_request.rs	HTTP/1.1 framing, parsing, smuggling checks
bin/tcp_echo_server.rs	Blocking sockets, thread per connection
bin/http_server.rs	REST routing, keep-alive, status codes
bin/epoll_echo.rs	Readiness notification, partial writes, backpressure

All module paths are relative to rust-interview-lab/src/.

Contents

Fundamentals.....	4
Operating systems	4
Concurrency	5
Networking	6
Data structures and complexity.....	7
1. The execution model	8
2. Threads and safe concurrency	9
Running work on threads	9
Choosing how to share data	9
Bugs the compiler does not prevent.....	10
3. Smart pointers and interior mutability	11
4. Algebraic data types and idioms	12
5. Synchronization primitives	13
Mutex.....	13
Condition variables	13

Semaphores	14
6. Building a spin lock.....	15
7. Queues and backpressure.....	16
8. A lock-free ring buffer.....	17
9. Contention and sharding.....	18
10. Memory alignment and arena allocation	19
11. Async Rust.....	20
Pin.....	20
An executor.....	20
Blocking inside async code	21
Async runtimes and readiness	21
12. I/O multiplexing.....	22
Level-triggered and edge-triggered notification	22
Partial writes.....	22
Errors and shutdown	22
13. TCP for server developers.....	24
14. HTTP/1.1 and REST	26
Message framing	26
Persistent connections.....	26
REST method semantics	27
15. Retries and consistent hashing	28
Retries.....	28
Consistent hashing	28
16. How the pieces fit together.....	29
Summary	30

Fundamentals

This section gives a one- or two-sentence definition of each concept that interviews commonly check. Later sections explain the mechanisms in depth. The programs `os_paging`, `os_scheduler`, `concurrency_deadlock`, `concurrency_amdahl`, `concurrency_false_sharing`, `net_ipv4`, `net_window`, `cs_fib`, and `cs_locality` in `rust-interview-lab/src/bin/` demonstrate several of them.

Operating systems

Concept	Definition
Process and thread	A process has its own address space. Threads in a process share it, and each has its own registers and stack
Context switch	The kernel saves one task's registers and restores another's. It takes microseconds and invalidates CPU cache and TLB contents
User and kernel space	User code cannot access hardware or kernel memory directly; it asks the kernel through system calls
System call	A controlled entry into the kernel. Registers carry the call number and arguments; failures return an error code (<code>errno</code>)
Virtual memory	Per-process page tables map virtual addresses to physical frames, providing isolation and lazy allocation
Page fault	An access to a page that is not mapped, or not mapped with the needed permissions. Minor faults need no disk read; major faults do
Copy-on-write	After fork, parent and child share pages read-only, and a page is copied only when one of them writes it
fork and exec	<code>fork</code> creates a child process; <code>exec</code> replaces the current program and keeps the process ID
Scheduling	The kernel runs and preempts tasks based on priority and fairness, using a timer interrupt
File descriptor	A small per-process integer that refers to an open file, socket, or pipe. 0, 1, and 2 are standard input, output, and error
Buffering	The C standard library buffers output in user space; unflushed output is lost on <code>_exit</code> or a crash
Interrupts and polling	An interrupt stops the CPU to run a handler; polling checks a device's status periodically

`02-linux-concepts.md` covers these topics in depth.

Concurrency

Concept	Definition
Race condition	A result depends on timing because threads access shared state without synchronization
Critical section	Code that must run with mutual exclusion
Mutex	A lock that puts waiting threads to sleep. The default choice
Spin lock	A lock whose waiters loop on an atomic. Suitable only for very short critical sections without oversubscription
Semaphore	A count of permits. It limits concurrency rather than protecting data
Condition variable	Lets a thread wait until a condition is true. The condition is checked in a loop, because wakeups can be spurious
Atomic operation	A read-modify-write that other threads cannot interrupt, such as <code>fetch_add</code> or <code>compare_exchange</code>
Deadlock	Requires mutual exclusion, hold-and-wait, no preemption, and circular waiting. Removing any one prevents it
Livelock and starvation	Livelock: threads keep working without progress. Starvation: a thread never gets the resource
Thread pool	A fixed set of worker threads that take tasks from a queue
Producer-consumer	A bounded queue between producers and consumers; the bound provides backpressure
Amdahl's law	The maximum speedup from parallelism is limited by the fraction of work that must run serially
False sharing	Unrelated variables on the same cache line slow each other down because writes move the line between cores

Networking

Concept	Definition
Layers	Link, internet, transport, and application. Each adds addressing or reliability
TCP and UDP	TCP provides an ordered, reliable byte stream over a connection; UDP sends individual datagrams without delivery or ordering guarantees
Three-way handshake	SYN, SYN-ACK, ACK. It exchanges initial sequence numbers and confirms both directions work
Connection teardown	Each side sends a FIN, which the other acknowledges. The side that closes first waits in <code>TIME_WAIT</code>
Flow and congestion control	The receive window protects the receiver; the congestion window protects the network. The sender uses the smaller
Socket API	socket, bind, listen, accept, connect. A TCP connection is identified by source and destination addresses and ports
Blocking and non-blocking I/O	A blocking read waits for data; a non-blocking read returns <code>EAGAIN</code> immediately when no data is available
DNS	Resolves names to addresses, usually over UDP, with caching controlled by TTLs
HTTP	Requests and responses over TCP. Methods have safety and idempotency semantics; bodies are framed by Content-Length or chunked encoding
TLS	Encrypts and authenticates a connection between TCP and the application. The handshake adds round trips
NAT	Rewrites addresses and ports so that many hosts share one public address
Latency and bandwidth	Latency is the delay of each round trip; bandwidth is the volume per second. More bandwidth does not reduce latency
Load balancing	Distributes connections across backends. Consistent hashing keeps most keys on the same backend when backends change

Data structures and complexity

Concept	Definition
Big-O	How cost grows with input size, ignoring constant factors and lower-order terms
Amortized cost	An operation that is occasionally expensive but cheap on average, such as <code>Vec::push</code>
Array and linked list	Arrays are contiguous and cache-friendly; linked lists allow $O(1)$ splicing but require pointer chasing
Hash map	$O(1)$ expected lookup, $O(n)$ in the worst case with many collisions; unordered
Heap	$O(1)$ peek and $O(\log n)$ push and pop; used for top-k and priority scheduling
Balanced tree	$O(\log n)$ lookup, with ordered iteration and range queries
Stack and queue	Last-in first-out and first-in first-out; the basis of DFS and BFS
Recursion and iteration	Recursion uses the call stack, so convert deep or unbounded recursion to iteration
Memoization	Caching the results of subproblems, which can turn exponential recursion into polynomial time
Cache locality	Sequential memory access is much faster than random access, because data moves between memory and CPU caches in cache-line-sized blocks
Idempotency	Repeating an operation has the same effect as performing it once, which makes retries safe
CAP	During a network partition, a distributed system must give up either consistency or availability

1. The execution model

A process has its own address space. A thread shares the address space, file descriptor table, and signal handlers with the other threads in its process. On Linux, both are represented by the same kernel structure, `task_struct` (see `02-linux-concepts.md`). Threads are therefore cheaper to create than processes, but switching between them is still far more expensive than a function call.

The kernel offers two ways to wait for I/O:

- **Blocking.** `read` returns when data is available, and the thread does not run while it waits. The code is sequential, and the kernel manages the waiting.
- **Non-blocking.** `read` returns `EAGAIN` immediately if no data is available. The program decides what to do next and must track the state of each operation itself.

These two options lead to two server designs:

Model	How it waits	Strength	Cost
Thread per connection	Blocking calls	Simple sequential code; a failure affects one connection	One stack per connection, context switches, and a practical limit on concurrent connections
Event loop	Non-blocking calls and a readiness API such as <code>epoll</code>	One thread can serve many connections	Per-connection state machines, partial reads and writes, and more complex control flow

`bin/tcp_echo_server.rs` implements the first model, and `bin/epoll_echo.rs` the second. Neither is better in general. A bounded thread pool is a good fit for blocking work such as disk access or DNS lookups. An event loop is a good fit for tens of thousands of mostly idle connections, where a thread stack for each would use too much memory.

The kernel can also report I/O in two ways:

- **Readiness (the reactor pattern).** The kernel reports that a socket can be read or written, and the program performs the operation. `select`, `poll`, and `epoll` work this way.
- **Completion (the proactor pattern).** The program submits an operation, and the kernel reports when it has finished. `io_uring` and Windows IOCP work this way.

The two lead to different code. With readiness, the program reads or writes until it gets `EAGAIN` and manages its own buffers. With completion, the program hands a buffer to the kernel and receives a notification when the operation is done. Linux development has moved toward completion interfaces with `io_uring`, but most production runtimes, including Tokio, still use readiness.

2. Threads and safe concurrency

Rust guarantees that safe code cannot cause a data race. Two marker traits enforce the guarantee. Both are *auto traits*: the compiler implements them for a type whose fields all implement them.

- **Send**: a value can be moved to another thread.
- **Sync**: a shared reference `&T` can be used from several threads. Equivalently, `T` is **Sync** if `&T` is **Send**.

The types that do not implement these traits show what they protect against. `Rc<T>` is neither **Send** nor **Sync**, because its reference count is not atomic, and `RefCell<T>` is not **Sync**, because its borrow flag is not atomic. Code that moves an `Rc` into `thread::spawn` does not compile, so a potential data race is rejected before the program runs. When a type is thread-safe only because of rules the compiler cannot check, its author writes `unsafe impl Send` or `unsafe impl Sync`, and a safety comment must state those rules.

Running work on threads

`problems/threads.rs` shows three approaches, in increasing order of setup:

- `thread::scope` runs threads that can borrow local data, because all of them are joined before the scope returns. Workers can borrow a slice directly, with no `Arc` and no `'static` bound, and it is impossible to forget a join.
- `thread::spawn` runs a thread that must own its data, because it may outlive the caller. The closure must be `'static`, so shared data is passed in an `Arc`, and the result is returned through the `JoinHandle`.
- **A thread pool** reuses threads for many small tasks. Creating a thread for each short task costs more than the task itself.

`thread::available_parallelism()` returns the number of threads the program can run in parallel. Use it to choose a pool size or the number of chunks to split work into.

Choosing how to share data

Need	Tool
Read-only data used by workers	<code>thread::scope</code> with <code>&T</code>
A counter	<code>AtomicUsize</code> with <code>fetch_add</code>
Several values that must change together	<code>Mutex<T></code> or <code>RwLock<T></code>
Passing ownership of data to another thread	An <code>mpsc</code> channel
Initializing a value once	<code>OnceLock<T></code>

The choice between an atomic and a mutex is about correctness before performance. An atomic keeps one value consistent. A mutex keeps an invariant across several values that

must change together. If you replace one mutex with several atomics, one update becomes several separate updates, and other threads can observe the state between them.

Bugs the compiler does not prevent

Rust prevents data races, not all concurrency bugs. These remain:

- **Deadlocks.** Two threads acquire two locks in opposite orders, or one thread locks a non-reentrant mutex it already holds. A consistent lock order, and releasing one lock before acquiring another where possible, prevent both.
- **Lost updates.** A thread reads a value under a lock, releases the lock, computes a new value, and writes it back under the lock. Another thread's update in between is lost. Hold the lock for the whole read-compute-write sequence.
- **Blocking an async runtime.** A blocking call inside an async task stops other tasks (section 11).
- **Unbounded growth.** A channel or queue without a bound lets a slow consumer cause unbounded memory use (section 7).

3. Smart pointers and interior mutability

By default, a Rust value has one owner, and mutation requires exclusive access. Smart pointers add shared ownership or shared mutation explicitly, and the type states which one:

Type	Ownership	Mutation	Across threads
<code>Box<T></code>	single	through <code>&mut</code>	if <code>T: Send</code>
<code>Rc<T></code>	shared, non-atomic count	no	no
<code>Arc<T></code>	shared, atomic count	no	if <code>T: Send + Sync</code>
<code>RefCell<T></code>	single	runtime checked	no
<code>Mutex<T></code>	single	blocking lock	yes
<code>Weak<T></code>	non-owning	no	as <code>Arc</code>
<code>Cow<'a, T></code>	borrow or own	no	as the borrow

Use `Rc` with `RefCell` for graphs, trees, and caches used by one thread, and `Arc` with `Mutex` for state shared between threads. Using `Rc` where `Arc` is required is a compile error.

- `Box` makes recursive types possible. An enum that contains itself directly would have infinite size, so the recursive variant holds a `Box`, which has a fixed size.
- `RefCell` moves the borrow check from compile time to run time. Calling `borrow_mut` while another borrow is active panics, so keep borrows short, and do not hold one while calling code that might borrow the same cell.
- `Weak` breaks reference cycles. If two `Rc` values point at each other, neither count reaches zero, and neither is freed. In a tree, parents hold children with `Rc` and children refer to parents with `Weak`, so an unreachable tree is freed. `TreeNode` in `problems/smart_pointers.rs` uses this design, and a test checks that after the parent is dropped, the child's `parent_value()` returns `None`.
- `Cow` suits a function that usually returns its input unchanged. `normalize` returns a borrowed `&str` when no change is needed and allocates a `String` only when it modifies the input, so the common case does not allocate.

4. Algebraic data types and idioms

`problems/adt_idioms.rs` implements several short patterns that each prevent a class of bugs.

A newtype for validated values. `GpuCount` wraps a `u32`. Its constructor, `GpuCount::new`, returns an error for values outside `1..=64`, and the field is private, so an invalid `GpuCount` cannot exist and later code does not need to check it again. `From<GpuCount> for u32` converts it back.

An enum for a closed set of commands. `Command` has the variants `List`, `Get`, `Create`, and `Delete`, and `Create` carries a name and a validated count. Every match on `Command` must handle every variant, so adding a variant produces a compile error at each place that must handle it.

Typed errors. `CommandError` distinguishes an empty line, an unknown verb, a missing argument, and an invalid count. A caller can handle one case specifically and print the others. An error represented as a `String` would force callers to parse it.

TryFrom at input boundaries. `impl TryFrom<u32> for GpuCount` validates the value where it enters the program. The rest of the code receives a `GpuCount` that is already valid.

Iterator adapters instead of index loops. `summarize` computes a count and a sum with one fold. There is no index variable to get wrong, and iterating directly over the slice needs no bounds checks.

`problems/state_machine.rs` applies the same idea to a workload lifecycle: the states are an enum, and an exhaustive match on `(current, next)` lists every allowed transition.

5. Synchronization primitives

Mutex

A mutex provides mutual exclusion. Rust's Mutex design enforces the rules for using it:

- **The data is inside the mutex.** The only way to access it is through the guard that `lock()` returns, so code that forgets to lock does not compile.
- **Dropping the guard releases the lock.** An early return or a panic cannot leave the mutex locked.
- **Poisoning records panics.** If a thread panics while holding the guard, later calls to `lock()` return an error. The caller can propagate the failure with `unwrap`, or recover the data with `into_inner` on the error when the invariant still holds.

`std::sync::Mutex` is not reentrant. A thread that locks a mutex it already holds deadlocks, often in a way that looks like an unrelated hang, for example when a method that holds the lock calls another method that locks it again.

Condition variables

A condition variable lets a thread sleep until a condition becomes true. It is always used with a mutex that protects the data the condition depends on. The correct pattern is a loop:

```
let mut state = mutex.lock().expect("poisoned");
while !predicate(&state) {
    state = condvar.wait(state).expect("poisoned");
}
// The predicate is now true and the lock is held.
```

Each part of the pattern has a reason:

- **The condition is checked while holding the mutex.** A notification therefore cannot arrive between the check and the call to `wait` and be missed.
- `wait` **releases the mutex while sleeping and reacquires it before returning**, so the thread that changes the state can take the lock and notify.
- **The loop rechecks the condition.** `wait` can return without a notification (a spurious wakeup), and another thread may have changed the state again before this thread reacquired the lock.

`problems/bounded_queue.rs` uses this pattern twice: consumers wait until the queue is not empty, and producers wait until it is not full.

Semaphores

A semaphore holds a number of permits. `acquire` takes a permit, waiting if none are available, and `release` returns one. A semaphore limits how many threads do something at once, such as calling a downstream service; it does not protect data.

In `problems/semaphore.rs`, `acquire_guard`, `try_acquire`, and `try_acquire_timeout` return a guard that releases the permit when dropped. Without a guard, an error path that returns early can forget to release its permit, and each such error permanently reduces capacity until no work can proceed.

Primitive	Question it answers
Mutex	May I access this data now?
Condition variable	Has this condition become true?
Semaphore	Is one of the N slots free?

A bounded channel combines a queue with a limit on its size, which is effectively a semaphore. Using one bounded queue keeps the two consistent; a separate queue and semaphore can drift out of step if one is updated without the other.

6. Building a spin lock

`problems/spin_lock.rs` implements a lock with a single `AtomicBool`. Implementing one makes the requirements of any lock explicit:

1. **Mutual exclusion.** At most one thread holds the lock, so the guard can safely provide `&mut T`.
2. **Visibility of previous writes.** A thread that acquires the lock must see every write the previous holder made. Each atomic operation is a synchronization point, so when the next thread observes the flag change, it also observes the writes made before the change.

The implementation works as follows:

- **Acquiring.** `compare_exchange_weak(false, true, Ordering::Acquire, Ordering::Relaxed)` sets the flag if it is clear. The weak variant can fail spuriously, which is harmless in a retry loop and lets it compile to a simpler instruction sequence on some architectures.
- **Waiting.** While the flag is set, the loop checks it with a `load(Ordering::Relaxed)` and calls `std::hint::spin_loop()`. Checking with a load before retrying the compare-and-swap avoids repeated failed writes, which would each invalidate the cache line on other cores. `spin_loop()` emits a CPU pause instruction, which reduces power use and helps the processor schedule the waiting loop.
- **Releasing.** The guard's `Drop` calls `store(false, Ordering::Release)`, so an early return or a panic still releases the lock.
- **Thread safety.** `unsafe impl<T: Send> Sync for SpinLock<T>` states that sharing the lock between threads is safe if `T` can be sent between threads, because the lock gives only one thread at a time access to `T`. `Send` is implemented automatically.

Memory orderings. The `Ordering` argument describes the consistency the operation requires. `SeqCst` makes all operations appear in one sequential order. `Relaxed` makes no ordering guarantee between operations. `Acquire` behaves like `SeqCst` for the load part of an operation, and `Release` like `SeqCst` for the store part. On x86-64 and aarch64 all orderings compile to the same instructions, so the choice documents intent more than it changes behavior. The usual guidance is to use `SeqCst` unless you can explain why a weaker ordering is correct.

A spin lock is the wrong choice in most code. A waiting thread occupies a CPU core without doing work. If there are more runnable threads than cores, the thread holding the lock may be descheduled while others spin, and every waiter makes no progress until it runs again. `std::sync::Mutex` spins briefly and then puts waiting threads to sleep, so use it unless the critical section is extremely short and threads are not oversubscribed.

7. Queues and backpressure

With an unbounded queue, a producer never waits. If items arrive faster than they are processed, the queue grows at the difference between the two rates until the process runs out of memory. A bounded queue makes the producer wait when the queue is full, which slows the producer to the consumer's rate and passes the slowdown back toward its source, where it can be handled.

`problems/bounded_queue.rs` implements `BoundedQueue<T>` with one mutex and two condition variables:

- `push` waits while the queue is full. If the queue has been closed, it returns `Err(item)`, which gives the item back to the caller instead of dropping it.
- `pop` waits while the queue is empty. It returns `None` once the queue is closed and empty, which ends a consumer's loop.
- `close` marks the queue closed and wakes all waiting producers and consumers, so no thread remains blocked during shutdown.

The `close` operation is as important as the capacity limit. Without it, a consumer waiting on an empty queue cannot learn that the producers have stopped, and shutdown would depend on timeouts.

Apply backpressure at every stage that can receive more work than it can process: the listener stops accepting connections, a connection stops reading requests, a queue makes producers wait, and clients honor `Retry-After`. Adding it at only one stage moves the unbounded growth to another stage.

8. A lock-free ring buffer

`problems/ring_buffer.rs` implements `SpSCRing<T, N>`, a fixed-capacity queue for exactly one producer thread and one consumer thread, without locks.

The ring keeps two atomic indexes. The producer is the only thread that writes `tail`, and the consumer is the only thread that writes `head`. Because each index has a single writer, neither needs a compare-and-swap:

- `push` reads its own `tail` with `Relaxed`, reads `head` with `Acquire` to check whether the ring is full, writes the item into the slot at `tail`, and then publishes the new `tail` with `store(..., Ordering::Release)`.
- `pop` reads its own `head` with `Relaxed`, reads `tail` with `Acquire` to check whether the ring is empty, takes the item from the slot at `head`, and then publishes the new `head` with `Release`.

The order of operations is what makes the unsafe slot access sound. The producer writes a slot before it advances `tail`, so the consumer only reads slots that have been fully written. The consumer advances `head` only after taking the item, so the producer never overwrites a slot the consumer has not read. The code documents these obligations in its safety comments.

The design depends on having one producer and one consumer. Two producers could both read the same `tail` and write the same slot, and two consumers could take the same item. A queue with several producers or consumers needs a different algorithm, and in most applications a `Mutex<VecDeque<T>>` is fast enough that a lock-free queue is not worth the added complexity.

9. Contention and sharding

A single mutex around a hash map makes every access wait for every other access, even when they involve different keys. `problems/sharded_cache.rs` divides the map into several shards, each a separate map with its own lock, and chooses the shard from the key's hash. Accesses to keys in different shards run in parallel.

The design has consequences:

- **Whole-map operations visit every shard.** `len` and `clear` lock each shard in turn.
- **There is no consistent snapshot.** Other threads can change one shard while `len` is counting another, so the result may never have been the exact size at any moment.
- **The shard count is fixed.** Changing it would move keys between shards, which needs a different design.
- **The shard index is a bit mask.** The constructor rounds the shard count up to a power of two, so `hash & (shard_count - 1)` selects the shard without a division.
- **with `avoids cloning`.** It runs a closure on a reference to the value while the shard's lock is held, which helps when values are large or expensive to clone.

False sharing is a related hardware problem. Two independent variables, such as two counters updated by different threads, can sit on the same 64-byte cache line. Each write invalidates the line in the other core's cache, so the threads slow each other down even though they never touch the same variable. Placing hot variables on separate cache lines, for example with `#[repr(align(64))]` padding, removes the effect. The problem is invisible in the source code but visible in `perf` measurements; `bin/concurrency_false_sharing.rs` demonstrates it.

10. Memory alignment and arena allocation

Every type has an alignment: its values must start at a memory address that is a multiple of that alignment. Accessing misaligned data is slower on some processors and invalid on others. An allocator must therefore return memory whose start address satisfies the requested alignment.

`problems/bump_allocator.rs` implements the simplest allocator that does so. It keeps a cursor into a byte buffer. For each allocation, it rounds the cursor up to the requested alignment, returns the memory at that position, and advances the cursor by the size. Each allocation is $O(1)$, and there is no per-allocation bookkeeping. `reset` frees everything at once by moving the cursor back to the start.

An arena cannot free individual allocations. If short-lived and long-lived objects share an arena, the space used by the short-lived objects cannot be reused until the whole arena is reset. Arenas are well suited to work where all objects have the same lifetime, such as the allocations made while handling one request or rendering one frame.

Alignment padding counts toward usage. After a one-byte allocation, a block with 16-byte alignment starts at the next 16-byte boundary, so up to 15 bytes of padding separate the two. `used()` reports the total including padding.

11. Async Rust

An `async fn` or `async block` compiles to a state machine that implements the `Future` trait. The trait has one method:

```
fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
```

The contract for `poll` has three parts:

- **A future does nothing until it is polled.** Calling an `async fn` only creates the future.
- `Poll::Ready(value)` means the future has finished. It must not be polled again.
- `Poll::Pending` means the future cannot make progress yet. Before returning `Pending`, the future must arrange for the `Waker` from `cx` to be called when progress is possible, for example by registering it with a timer or a socket.

A future that returns `Pending` without arranging a wakeup is never polled again, and the task hangs without any error. This is the most common mistake in hand-written futures.

Pin

`poll` takes `Pin<&mut Self>` because the state machine generated for an `async block` can contain references to its own fields, for example a reference to a local variable that is held across an `.await`. Moving the future in memory after it has started would leave those references pointing at the old location. `Pin` guarantees that the future will not be moved again. Types that are safe to move even while pinned implement the `Unpin` marker trait. Most ordinary types are `Unpin`; futures generated from `async blocks` generally are not.

An executor

`problems/async_mini.rs` implements the two basic parts of an `async` runtime:

- `block_on` pins one future and polls it in a loop. When the future returns `Pending`, the thread parks. The future's waker unparks the thread, so the loop polls again only when progress is possible, and the CPU is idle while it waits.
- `MiniExecutor` keeps a queue of tasks. Each task holds a future, and its waker puts the task back on the queue. `run` takes tasks from the queue and polls them until the queue is empty.

Tasks are stored in an `Arc`, because both the queue and every waker created for the task refer to it. In the implementation, `wake` and `wake_by_ref` both push a clone of the `Arc` onto the queue, which increments a reference count rather than copying the future.

Blocking inside async code

A blocking call inside an async task stops every other task scheduled on the same thread until the call returns. Move blocking work to a separate thread pool, with `spawn_blocking` in Tokio. A blocked thread in that pool uses one thread's stack instead of stalling the runtime.

On a multithreaded runtime, a spawned future must be `Send`, which requires every value held across an `.await` to be `Send`. A `std::sync::MutexGuard` is not `Send`, so holding one across an `.await` fails to compile. Release the guard before the `.await` by limiting its scope, or use `tokio::sync::Mutex` if the lock must be held while the task is suspended.

Async runtimes and readiness

An async runtime combines the readiness model from section 1 with a task queue. A task that reads a socket registers interest with the reactor and returns `Pending`. When `epoll` reports the socket readable, the reactor calls the task's waker, the waker puts the task back on the queue, and the executor polls it again. Understanding `epoll` therefore explains most of how an async runtime behaves.

12. I/O multiplexing

`select`, `poll`, and `epoll` report which file descriptors are ready for reading or writing.

Interface	Cost	Limitation
<code>select</code>	$O(n)$ per call, and the descriptor sets are copied in and out on each call	Descriptor numbers must be below <code>FD_SETSIZE</code> , usually 1024
<code>poll</code>	$O(n)$ per call	No fixed limit on descriptors
<code>epoll</code>	$O(\text{number of ready descriptors})$ per wait	Linux only

`epoll` keeps the list of registered descriptors in the kernel, so registering a descriptor (`epoll_ctl`) is separate from waiting for events (`epoll_wait`), and each wait does not have to pass the whole list. `kqueue` is the equivalent on BSD and macOS.

Level-triggered and edge-triggered notification

- **Level-triggered** (the default) reports a descriptor as ready for as long as the condition holds. A socket with unread data is reported on every call to `epoll_wait`.
- **Edge-triggered** (`EPOLLET`) reports only when the state changes, such as when new data arrives. The program must read until `EAGAIN` after each notification, because it will not be notified again about data that was already buffered.

Edge-triggered mode generates fewer notifications, and it is easier to get wrong. If a handler stops reading before `EAGAIN`, the remaining data stays in the socket buffer with no further notification, and the connection stalls. The bug often appears only under load, when more data arrives at once.

Partial writes

A non-blocking write can accept fewer bytes than it was given. `bin/epoll_echo.rs` keeps an output buffer for each connection and records how many bytes have been written. It registers interest in `EPOLLOUT` only while the buffer holds unwritten bytes, and removes that interest once the buffer is empty. Registering `EPOLLOUT` permanently would wake the loop constantly, because an idle socket is almost always writable.

The alternative, looping until the socket has accepted every byte, blocks the event loop while one slow client reads its data, and every other connection waits. This is the most common defect in hand-written event loops, and it causes incorrect behavior, not just lower performance.

Errors and shutdown

- `EAGAIN` **or** `EWOULDBLOCK` means the operation would block; try again when the descriptor is ready. It is not a failure.
- `EINTR` means a signal interrupted the call; retry it.

- EPIPE is returned by a write to a socket whose peer has closed. The kernel also sends SIGPIPE, whose default action terminates the process, so servers ignore SIGPIPE and handle EPIPE as an error. `bin/epoll_echo.rs` sets SIGPIPE to SIG_IGN for this reason.
- **Graceful shutdown** stops accepting new connections, stops reading new requests, lets in-progress requests finish, and closes idle connections. Connections still open after a deadline are closed. Without a deadline, one client that never finishes can prevent shutdown from completing.

13. TCP for server developers

Many production problems with TCP come from connection state transitions rather than from data transfer.

Connection setup. When a client sends SYN, the kernel records the half-open connection in the *SYN queue*. When the handshake completes, the connection moves to the *accept queue*, where it waits for the application to call `accept`. The backlog argument to `listen`, capped by `net.core.somaxconn`, sets the size of the accept queue.

- If the accept queue is full, the kernel drops the final ACK of new handshakes, and clients retransmit and eventually time out. A server that calls `accept` too slowly produces connection failures that look like a network problem. `nstat -az TcpExtListenOverflows` counts these events.
- If the SYN queue is full, for example during a SYN flood, the kernel can use *SYN cookies* (`net.ipv4.tcp_syncookies`), which encode the connection state in the sequence number instead of storing it.

Connection teardown. Closing a connection sends FIN, and the peer acknowledges it. The side that closes first enters `TIME_WAIT` for twice the maximum segment lifetime (60 seconds on Linux), so delayed packets from the old connection are discarded rather than delivered to a new connection that reuses the same addresses and ports.

Half-close. `shutdown(SHUT_WR)` sends FIN but keeps the connection open for reading. A peer that has finished sending may still be waiting to receive a reply, so a server that treats the end of the client's input as the end of the connection can cut off its own response.

Socket options that often matter:

Option	Effect
<code>SO_REUSEADDR</code>	Allows binding a listening port while old connections on it are in <code>TIME_WAIT</code> , so a restarted server can listen immediately
<code>SO_REUSEPORT</code>	Allows several sockets to bind the same address and port; the kernel distributes incoming connections among them
<code>TCP_NODELAY</code>	Disables Nagle's algorithm, which delays small writes in order to combine them into fewer packets
<code>SO_KEEPALIVE</code>	Enables TCP keepalive probes on idle connections; the default idle time before probing is two hours

Nagle's algorithm reduces the number of packets at the cost of latency. Combined with delayed acknowledgments on the receiver, it can add around 40 milliseconds to request-response exchanges, so interactive protocols and RPC systems set `TCP_NODELAY`.

To detect a peer that has disappeared, applications usually send their own heartbeats with a response deadline. This is more predictable than relying on TCP keepalive, whose defaults are far too slow and whose settings are system-wide unless set per socket.

14. HTTP/1.1 and REST

Message framing

An HTTP/1.1 message consists of a start line, header fields, an empty line, and an optional body. Determining where the body ends is the part of parsing with security consequences.

- Content-Length gives the exact number of bytes in the body.
- Transfer-Encoding: chunked sends the body as a series of chunks, each preceded by its length, and ends with a zero-length chunk. It is used when the length is not known in advance.
- **A request with neither header has no body.**

Two framing rules prevent **request smuggling**:

1. **Reject a message that has both** Content-Length **and** Transfer-Encoding. When a proxy and a backend server choose different headers to determine the body length, the bytes one of them considers part of the body are parsed by the other as the start of a second request. An attacker can use this to send a request that bypasses the proxy's checks.
2. **Reject a message with several** Content-Length **values that differ**. Some parsers accept identical duplicates, but rejecting all duplicates is simpler and safer.

`problems/http_request.rs` enforces both rules. It returns a typed `ParseError` with the variants `Incomplete`, `Malformed`, `MissingHost`, `AmbiguousBodyLength`, `UnsupportedTransferEncoding`, and `PayloadTooLarge`. `Incomplete` is not a failure: it means the buffer does not yet contain a whole request, so the caller reads more bytes and parses again.

The parser performs two more checks:

- **Exactly one** Host **header**. HTTP/1.1 requires it, and a request without one is rejected.
- **No obsolete line folding**. A header line that begins with a space or tab continues the previous header in an old syntax that implementations interpret differently, so the parser rejects it.

The parser also enforces size limits on headers and bodies. Without limits, a client can consume memory, or hold a connection open indefinitely by sending a few bytes at a time.

Persistent connections

HTTP/1.1 connections stay open for further requests by default; HTTP/1.0 connections stay open only with `Connection: keep-alive`. A server reads one request after another from the same connection. It closes the connection when the client sends `Connection:`

close, when an HTTP/1.0 client did not ask for keep-alive, or after an error that leaves the position in the byte stream uncertain.

Responses on one connection must be sent in the order the requests arrived. *Pipelining*, in which a client sends several requests without waiting for responses, is rarely supported, because a slow first request delays every response behind it (head-of-line blocking). `bin/http_server.rs` handles one request at a time on each connection.

REST method semantics

Clients, proxies, and caches rely on each method's defined properties:

Method	Safe	Idempotent
GET, HEAD, OPTIONS	Yes	Yes
PUT, DELETE	No	Yes
POST	No	No
PATCH	No	Not necessarily

A *safe* method does not change server state. An *idempotent* method has the same effect whether it is sent once or several times.

`bin/http_server.rs` returns 405 Method Not Allowed for a known path with an unsupported method, and 404 Not Found for an unknown path. Returning 404 in both cases would tell a client that a valid path does not exist, which makes client bugs harder to find.

For methods that are not idempotent, an idempotency key sent with the request lets the server recognize a retry and return the original result instead of performing the operation again. This is what makes client retries of POST requests safe, and it must be designed into the API; a client retry loop cannot provide it alone.

15. Retries and consistent hashing

Retries

`problems/retry.rs` separates the retry policy from the operation being retried:

- **Exponential backoff with a cap.** The delay doubles after each attempt, up to a maximum, and the number of attempts is limited, so the total time spent retrying is bounded.
- **Full jitter.** The actual delay is chosen uniformly between zero and the backoff value. Without jitter, clients that failed at the same moment retry at the same moment and recreate the load spike that caused the failure.
- **Retry-After as a minimum.** When the server says how long to wait, the client waits at least that long.
- **Only retryable errors.** A request rejected as `InvalidRequest` or `NotFound` fails the same way every time, so retrying it only consumes the retry budget.

Two further rules prevent retries from making an outage worse. Retry only idempotent operations, or operations sent with an idempotency key. And limit the total retry effort with a deadline or an attempt budget, so that a failing dependency does not cause work to build up in every client.

`retry` takes the random source as a closure parameter, `unit_random`, which returns a value in $[0, 1)$. Tests pass a fixed value to make delays deterministic, and production code passes a real random number generator.

Consistent hashing

Assigning each key to $\text{hash}(\text{key}) \% N$ moves almost every key when N changes. A cache cluster that adds one server loses most of its cached entries, and a sharded queue moves most of its work.

`problems/consistent_hash.rs` places each node at several points on a hash ring and assigns each key to the first node point at or after the key's hash. When a node is added, it takes over only the keys between its points and the preceding points; when a node is removed, only its keys move. Each node is placed at replicas points, called *virtual nodes*, because a single point per node divides the ring unevenly; more virtual nodes produce a more even distribution.

The module's test checks the defining property: after a node is added, every key either stays with its previous node or moves to the new node.

16. How the pieces fit together

A request to a GPU control plane service uses most of these mechanisms in sequence:

1. **Accept.** The listener accepts the connection. `SO_REUSEADDR` let the server restart and listen while old connections were in `TIME_WAIT`, and the accept queue absorbs bursts of new connections.
2. **Parse.** The server parses the request with size limits, rejects ambiguous framing, and reads more data when the request is incomplete.
3. **Route.** The router selects a handler, and the method's semantics determine whether a client can safely retry.
4. **Limit concurrency.** The handler acquires a semaphore permit, so the amount of concurrent work stays within a set limit, and it may read from a sharded cache.
5. **Retry.** Calls that can fail transiently use the retry policy, with jitter from the random source and `Retry-After` as a minimum delay.
6. **Distribute.** Calls to backends use consistent hashing, so a change in backend membership moves only a small share of keys.
7. **Respond.** The response is written through the connection's output buffer. A partial write leaves the rest buffered until the socket is writable, so one slow client does not stall the server.
8. **Shut down.** Closing the work queue wakes blocked producers and consumers, and the event loop stops accepting connections and drains in-progress work before a deadline.

Each step has a characteristic failure, and each module in this chapter addresses one of them. Production systems use the same structure at larger scale.

Summary

- Choose between thread-per-connection and event-loop servers based on the number of connections and the kind of work, and between readiness and completion APIs based on the platform.
- Send and Sync prevent data races at compile time, but deadlocks, lost updates, blocked runtimes, and unbounded queues remain your responsibility.
- Use atomics for single values, mutexes for invariants across several values, condition variables in a loop, and semaphores with guards.
- Bound every queue, and provide a way to close it so shutdown cannot hang.
- A future must arrange a wakeup before returning Pending; an executor is a task queue driven by wakers and readiness events.
- In event loops, handle partial writes with per-connection buffers and register for EPOLLOUT only while data is pending.
- Reject ambiguous HTTP framing to prevent request smuggling, and make retries safe with idempotency, backoff, jitter, and budgets.

Source: [14-concurrency-async-networking.md](#) · [PDF](#) · [All study guides](#)